

Ordinal Arithmetic: An Ordinal-First Metalanguage Approach to Definable Arithmetic Truth

Tyler Roost

Manuscript draft, September 10, 2026

Abstract

Ordinatics proposes an arithmetic of ordinal forms with division and a finite-valued wrap map, while retaining distinctions between construction paths and numerical outcomes. We give a bounded mathematical reconstruction directed at the definition of arithmetic truth in an ordinal framework. Finite ordinals supply the arithmetic domain, and ordinals below ω^ω index a hierarchy of languages whose truth predicates apply to earlier languages. In an explicit set-theoretic metatheory, the hierarchy has a unique satisfaction interpretation and preserves arithmetic truth under passage to later stages. Each stage nevertheless lacks a definition of its own full truth predicate. We also construct a rational-function value layer with partial specialization $W(X) = -1/2$. To formalize the stronger objective of Gödelian completeness through fractal meta-representations, we specify a ranked self-closing meta-surface in which closed derivations and their acceptance derivations can be reified at successive ranks. We formalize observation-exactness as an information-preservation condition, compare the target with Feferman’s completeness theorem for transfinite reflection progressions, and prove a conditional record-coverage theorem whose conclusion quantifies over native records alone. A checked finite calculus retains composed rule trees, encodes records and formulas as single free ground terms, and checks packed numerical representations. One explicit model of all current source clauses preserves those records, while its path relation does not implement their direct transition to conclusions. A full Hypermath-derived ranked realization, arithmetic checker agreement, arithmetic interpretation, and soundness proof remain open. The contribution is an explicit interface, bounded constructions, and proof-obligation analysis; it does not establish arithmetic completeness. An explicit instruction machine checks finite inference trees and their execution traces; native form closure alone cannot certify those records. Two models also show that the current self-derivation target is independent of the formalized clauses and does not supply the required arithmetic bridges.

Keywords: Ordinatics; ordinal arithmetic; satisfaction; ramified truth; uniform reflection; fractal meta-representation; derivation paths.

1 The foundational question

What would it mean to make ordinal arithmetic the underlying foundation of a Tarski-definable arithmetic? The phrase needs a language and a domain of definability. In this paper it means that the truth and satisfaction relations for an arithmetic object language are definable in a specified richer framework, with explicit ordinal bounds on the languages to which each truth predicate applies. It does not mean decidability of arithmetic or an unrestricted truth predicate definable inside its own language.

Here “ordinal-first” refers to the arithmetic domain and the ordering of semantic dependencies. The ambient metalanguage is the set theory specified below; the paper does not assert definability of arithmetic truth in the ordinal-arithmetic structure alone.

We use *Ordinalics* as the name of the overall framework. The motivating notes distinguish a layer of ordinary ordinal operations, an intended extension admitting reciprocals and fractional powers, and a further complex-valued extension [17]. We retain these as distinct layers within Ordinalics. They also distinguish derivation-path identity, written = in those notes, from outcome congruence $\equiv$ and continuation similarity $\sim$. This separation suggests that an expression should retain more information than its numerical image. We preserve that design question while making the mathematical interpretation explicit.

Three roles of ordinals must be distinguished. First, finite ordinals carry ordinary arithmetic. Second, transfinite ordinals organize the dependencies of semantic definitions. Third, ordinal expressions can motivate algebraic expressions that admit division. These roles can coexist in a typed framework, but their operations need not agree. An ordinal rank is not automatically a field value, and a field value is not a truth predicate.

The established background is the definition of truth through satisfaction and the restriction on truth definitions for semantically closed languages [1]. Typed and ordinal-indexed truth hierarchies are also established constructions [2]. Feferman’s work studies transfinite progressions of axiomatic theories [5], and a modern proof and complexity analysis makes the associated arithmetic completeness theorem explicit [6]. Iterated Tarskian truth, induction, and reflection have been connected directly in proof-theoretic analyses [3], [4]. Recursive and cyclic representations of proofs with global correctness conditions form another developed literature [10], [11]. Recent Reflective Grounded Arithmetic gives machine-checked same-language proof machinery and internal truth under paracomplete grounded semantics [8], [9]. Our construction below begins as a classical semantic hierarchy, not an identification with any of those systems. No priority claim is made for ordinal-indexed truth hierarchies, completeness of reflection progressions, recursive proof graphs, internal proof checking, polynomial fields, or localization. The project-specific open target is a sound bridge from the native path-retaining record calculus to a named reflection progression covering standard classical arithmetic truth. Establishing its novelty requires comparison with these adjacent systems as well as completion of the bridge.

Ambient assumptions. We work in ordinary classical mathematics formalizable in Zermelo–Fraenkel set theory with the Axiom of Choice, abbreviated ZFC. Its operational role is to supply sets, finite syntax, and transfinite recursion, not a numerical unit. All quantities below are mathematical objects with no physical dimension. The use of this metatheory is explicit: the paper does not derive its set existence assumptions from the source notes’ $\square$ ground-and-term-former. A faithful internal derivation in that system remains a separate research obligation.

2 Ordinal arithmetic without a change of operations

Let ω be the least infinite ordinal. Write $+_o$ and $\cdot_o$ for ordinary ordinal addition and multiplication, defined recursively in the right argument. In particular,

$$\begin{aligned} \alpha +_o 0 &= \alpha, & \alpha +_o (\beta + 1) &= (\alpha +_o \beta) + 1, \\ \alpha \cdot_o 0 &= 0, & \alpha \cdot_o (\beta + 1) &= (\alpha \cdot_o \beta) +_o \alpha. \end{aligned}$$

At a nonzero limit, take the supremum of the earlier values. This is the standard ordinal recursion construction [12]. It gives

$$1 +_o \omega = \omega, \quad \omega +_o 1 > \omega, \quad 2 \cdot_o \omega = \omega, \quad \omega \cdot_o 2 = \omega +_o \omega.$$

Thus even the expression “twice an ordinal” needs an operand order.

Proposition 2.1 (Absorption obstructs a unital additive field interpretation). *Suppose a domain contains $1, \omega$ and $1 +_o \omega = \omega$. There is no map f into the additive group of a nontrivial field such that $f(1) = 1$ and $f(\alpha +_o \beta) = f(\alpha) + f(\beta)$ on these inputs.*

Proof. The absorption equation would imply $1 + f(\omega) = f(\omega)$. Cancellation gives $1 = 0$, contradicting nontriviality. $\square$

This obstruction does not depend on injectivity or on the field being Archimedean. Passing to an ordered field with infinite elements does not repair it. One may introduce new field operations on representations of ordinals, but that is a change of operations and must be marked as such.

For a concrete set-sized ordinal domain, fix

$$\theta = \omega^\omega, \quad \mathcal{O} = \{\alpha : \alpha < \theta\}, \quad \mathcal{A} = (\mathcal{O}, 0, 1, \omega, <, +_o, \cdot_o).$$

Every element has a unique finite ordinal polynomial representation

$$\alpha = \omega^k a_k +_o \cdots +_o \omega a_1 +_o a_0, \quad a_j \in \mathbb{N},$$

where coefficients multiply on the right and the leading coefficient is nonzero unless $\alpha = 0$. Finite addition and multiplication of these ordinals remain below θ : the highest exponent of a sum is at most the maximum of the input exponents, and that of a nonzero product is at most their sum. This representation is the restricted Cantor normal form. It gives computable finite codes and a computable comparison relation for the ranks used below. The bound is a convenient concrete choice, not a proposed ultimate ordinal of the theory.

Proposition 2.2 (Arithmetic as a definable finite-ordinal part). *In $\mathcal{A}$, the formula $x < \omega$ defines a domain whose induced $0, 1, +_o, \cdot_o$ structure is the standard arithmetic structure $(\mathbb{N}, 0, 1, +, \cdot)$.*

Proof. The ordinals below ω are exactly the finite ordinals. The displayed recursions restrict to the usual addition and multiplication recursions on them. Relativize every arithmetic quantifier to $x < \omega$. Induction on terms and formulas shows that this translation preserves satisfaction. $\square$

Consequently an ordinal foundation retains arithmetic’s capacity to encode syntax. Replacing natural-number notation with ordinal notation does not by itself remove diagonalization. The proposition establishes a definable arithmetic domain; it does not establish a definable truth predicate in $\mathcal{A}$.

3 A bounded value layer for Ordinatrics

The source wrap anchor $W(\omega) = -1/2$ is useful as a proposed numerical evaluation. To interpret it coherently, introduce an indeterminate X distinct from the ordinal ω . Let

$$K = \mathbb{Q}(X), \quad c = -\frac{1}{2}, \quad S = \{q \in \mathbb{Q}[X] : q(c) \neq 0\}, \quad R = S^{-1}\mathbb{Q}[X].$$

Here K is the field of rational functions. The ring R consists of rational functions admitting a representation with denominator nonzero at c ; this is the standard localization construction [14]. Define

$$W : R \longrightarrow \mathbb{Q}, \quad W(p/q) = p(c)/q(c).$$

We call K a *bounded value model of Ordinalics*: it realizes division in a value layer motivated by ordinal forms. It is not asserted to be the full Subreal type of the notes, an algebraically closed field, or a model of every $\square$ axiom.

Theorem 3.1 (Scope of wrap specialization). *The map W is a well-defined surjective unital ring homomorphism on R . Its kernel is $(X - c)R$. It cannot extend to a unital field homomorphism $K \rightarrow \mathbb{Q}$. An element $r \in R$ is a unit of R (has a multiplicative inverse in R) if and only if $W(r) \neq 0$.*

Proof. If $p/q = p'/q'$ then $pq' = p'q$; evaluating at c and dividing by the nonzero denominators proves well-definedness. Evaluation respects sums and products and fixes rational constants, which also proves surjectivity. A polynomial vanishes at c exactly when it is divisible by $X - c$, giving the kernel assertion. In K , the nonzero element $X - c$ has an inverse. An extension would send $(X - c)(X - c)^{-1} = 1$ to $0 \cdot W((X - c)^{-1}) = 1$, a contradiction. Finally, if $r = p/q$ and $W(r) \neq 0$, then $p(c) \neq 0$ and $q/p \in R$. Conversely, an inverse in R would give $W(r)W(r^{-1}) = 1$. $\square$

This construction recovers $W(X^n) = (-1/2)^n$ for every integer n , including $W(X^{-1}) = -2$. It also exposes a necessary limitation: $2X + 1$ is a nonzero field element with wrap value zero, so its inverse has no wrap value under this specialization. Undefined evaluation is distinct from zero evaluation.

There is a faithful *representation map*

$$j : \mathcal{O} \longrightarrow \mathbb{N}[X] \subset K, \quad j(\omega^k a_k +_o \cdots +_o a_0) = a_k X^k + \cdots + a_0.$$

It is a bijection onto polynomials with nonnegative integer coefficients, by uniqueness of the normal form. Pulling polynomial addition and multiplication back along j gives commutative operations $\oplus, \otimes$ on $\mathcal{O}$. By construction,

$$j(\alpha \oplus \beta) = j(\alpha) + j(\beta), \quad j(\alpha \otimes \beta) = j(\alpha)j(\beta).$$

They agree with ordinary arithmetic on finite ordinals, but $1 \oplus \omega = \omega +_o 1$ whereas $1 +_o \omega = \omega$. These are the natural ordinal operations on this restricted domain; their distinction from ordinary ordinal operations is essential. The field is generated by $j(\mathcal{O})$ over $\mathbb{Q}$, but j is not an embedding for $+_o, \cdot_o$.

3.1 Fractional powers and branch data

If fractional monomials are added, choose a coherent branch explicitly. For instance, with $\ell = -\log 2 + i\pi$, the map

$$\chi : (\mathbb{Q}, +) \longrightarrow (\mathbb{C} \setminus \{0\}, \cdot), \quad \chi(r) = \exp(r\ell)$$

satisfies $\chi(r + s) = \chi(r)\chi(s)$ and $\chi(1) = c$. However, $\chi(1/3) = 2^{-1/3}e^{i\pi/3}$ is not the real cube root $-2^{-1/3}$. Choosing a real cube root at $1/3$ and the principal value $2^{-1/6}e^{i\pi/6}$ at $1/6$ violates $\chi(1/3) = \chi(1/6)^2$. A mixed root convention is therefore not a single multiplicative character. Also, a collection of nonzero monomial values is not itself a field: it does not contain zero, let alone automatically supply all sums.

For complex z , the same expression $\Phi(z) = \exp(\ell z)$ is surjective onto $\mathbb{C} \setminus \{0\}$, but

$$\Phi(z) = \Phi(z') \iff z - z' \in \frac{2\pi i}{\ell} \mathbb{Z}.$$

This follows from the kernel of the complex exponential. Thus Φ induces a bijection from the quotient by that lattice onto $\mathbb{C} \setminus \{0\}$. Restricting its domain to the half-open strip $-\pi < \text{Im}(\ell z) \leq \pi$ also gives a set bijection onto $\mathbb{C} \setminus \{0\}$, but its inverse is discontinuous across the negative real axis. An analytic logarithm branch instead uses a slit codomain, for example $\mathbb{C} \setminus (-\infty, 0]$ [15]. A numerical image generally does not identify its source expression. Neither the real-root convention nor complex injectivity is needed for the truth construction below.

4 Expressions, outcomes, and information loss

Let $\mathcal{E}$ be finite typed expression trees. Ordinal constructors use $+_o, \cdot_o$; field constructors use $+_K, \cdot_K$ and inversion of nonzero field values; j is an explicit conversion on normalized ordinal values. An expression has its syntax tree, its typed denotation, and, where applicable, its wrap evaluation. These are different data.

The source relations motivate the following limited correspondence. Literal tree identity models one precise notion of path identity. Equal typed denotations model outcome congruence. Equal wrap values give a still coarser comparison on the wrap domain. This is a proposed mathematical interpretation, not a proof that these relations equal the source's full $=, \equiv, \sim$ filtration. In particular, continuation similarity has not been supplied with a transition system here. Nonempty overlap of continuation sets need not be transitive, so quotienting by it would require an additional argument.

Two short examples show why the distinctions matter. The ordinal expression trees for $1 +_o \omega$ and ω differ but have the same ordinal value. In contrast, the ordinals $\omega \cdot_o 2 +_o 1$ and 0 differ, while

$$W(j(\omega \cdot_o 2 +_o 1)) = W(2X + 1) = 0 = W(j(0)).$$

Neither syntactic identity nor ordinal identity can be reconstructed from this wrap value.

Lemma 4.1 (Ranks cannot be recovered from a collapsed evaluation). *If a map e identifies two ranks $a < b$, there is no irreflexive relation $\prec$ on its image satisfying $e(a) \prec e(b)$ whenever $a < b$.*

Proof. For the identified pair the required relation would read $e(a) \prec e(a)$, violating irreflexivity. $\square$

The operational consequence is specific: a wrap value alone cannot serve as a truth-level identifier if distinct comparable levels share it. Keep the original rank and the expression record alongside the value.

Regularized sums require a similar precaution. The ordinary ordinal sums $1 +_o 1 +_o \dots$ and $1 +_o 2 +_o 3 +_o \dots$ both have supremum ω , since their finite partial sums are unbounded finite ordinals. Any assignments of different regularized values therefore act on tagged summation expressions or additional analytic data, not solely on their resulting ordinal. This paper does not assume a summation regularization or derive a negative finite value for an ordinary divergent sum.

5 An ordinal-indexed language of arithmetic truth

Let $L_0 = \{0, 1, +, \cdot, <\}$ be the first-order language of arithmetic, with equality. Fix an injective effective coding into $\mathbb{N}$ of finite terms and formulas, including canonical finite-tuple codes for ordinal polynomial indices below θ . Choose the coding so that parsing, numeral formation, and

capture-avoiding substitution are computable, as with the usual coding of finite strings over an effectively indexed vocabulary. The symbol T_β is a unary predicate for each $\beta < \theta$. For $\alpha \leq \theta$, put

$$L_\alpha = L_0 \cup \{T_\beta : \beta < \alpha\}.$$

The predicate T_α will express truth for *sentences of* L_α ; it first occurs in $L_{\alpha+1}$ when that language is within the chosen range. At θ we construct the truth set externally without adding its symbol to L_θ .

Effective syntax, not effective truth. Ordinal polynomial codes can be checked and compared by finite algorithms. For each fixed $\alpha < \theta$, membership of a predicate index in L_α is decidable; at θ , all valid indices are allowed. Coding, substitution, and the formation of numerals can consequently be carried out arithmetically. This does not supply an algorithm for the truth sets. Computable level names and computable semantic contents are different assertions.

Write $\text{Sent}(L_\alpha)$ for codes of closed L_α formulas. At each stage the underlying arithmetic domain is the finite-ordinal domain $\mathbb{N}$. Truth predicates act on numbers that code sentences, not on ordinal values returned by W .

Definition 5.1 (Staged structures and truth sets). For $\alpha \leq \theta$, define recursively

$$M_\alpha = (\mathbb{N}, 0, 1, +, \cdot, <, (\text{Tr}_\beta)_{\beta < \alpha}), \quad (1)$$

$$\text{Tr}_\alpha = \{n \in \text{Sent}(L_\alpha) : M_\alpha \models \varphi_n\}, \quad (2)$$

where φ_n is the sentence coded by n . Numbers that do not code such a sentence are not in Tr_α .

This definition has two recursions. The outer recursion is on α . The inner satisfaction recursion is on formula construction with the interpretations of earlier predicates already fixed. Explicitly, for an assignment s of natural numbers to variables,

$$\begin{aligned} \text{Sat}_\alpha(t = u, s) &\iff t^{M_\alpha}[s] = u^{M_\alpha}[s], \\ \text{Sat}_\alpha(t < u, s) &\iff t^{M_\alpha}[s] < u^{M_\alpha}[s], \\ \text{Sat}_\alpha(T_\beta(t), s) &\iff t^{M_\alpha}[s] \in \text{Tr}_\beta \quad (\beta < \alpha), \\ \text{Sat}_\alpha(\neg\psi, s) &\iff \neg \text{Sat}_\alpha(\psi, s), \\ \text{Sat}_\alpha(\psi \wedge \eta, s) &\iff \text{Sat}_\alpha(\psi, s) \wedge \text{Sat}_\alpha(\eta, s), \\ \text{Sat}_\alpha(\exists x \psi, s) &\iff \exists n \in \mathbb{N} \text{ Sat}_\alpha(\psi, s[x \mapsto n]). \end{aligned}$$

Other connectives and the universal quantifier are defined from these. Satisfaction for open formulas is therefore included; the truth sets are their closed-sentence restriction.

Theorem 5.2 (Existence, uniqueness, and external definability). *In the stated metatheory, there is a unique hierarchy $(\text{Tr}_\alpha)_{\alpha \leq \theta}$ satisfying (1)–(2). Membership in this hierarchy and the associated satisfaction relations are definable in that metatheory from the fixed coding and ordinal bound.*

Proof. Given the sequence of truth sets below α , equation (1) specifies a set-sized structure in a set-sized language. The displayed formula recursion constructs its satisfaction relation: atomic clauses are fixed, and each composite clause uses only proper subformulas. Assignments may be taken to be finite sequences long enough for the formula's variables. The relevant formulas and assignments form a set, so this recursion defines a set relation. Separation then yields (2). This rule is a definable function of the earlier sequence. Transfinite recursion up to $\theta + 1$ gives existence. For uniqueness, a least stage of disagreement would have identical earlier predicates; formula induction gives identical satisfaction at that stage and hence identical truth sets, a contradiction. The recursive characterization is a set-theoretic definition of the sequence and its membership relation. $\square$

This establishes truth *in* the metatheory, not an algorithm deciding it and not truth *of* that metatheory. In a nonstandard model of the metatheory, the internal natural numbers and internal satisfaction construction must be distinguished from the intended external standard objects. No absoluteness across arbitrary models is claimed.

Theorem 5.3 (Stage compatibility and typed disquotation). *If $\alpha \leq \gamma \leq \theta$, the reduct of M_γ to L_α is M_α . Consequently L_α formulas have the same satisfaction value at these stages. If $\alpha < \gamma$, then for every L_α sentence φ ,*

$$M_\gamma \models T_\alpha(\ulcorner \varphi \urcorner) \leftrightarrow \varphi.$$

Here a code inside a formula denotes its standard numeral.

Proof. All structures have the same arithmetic part, and every earlier predicate keeps the set assigned at its defining stage. Term and formula induction proves the reduct assertion. The truth definition then gives the displayed equivalence. $\square$

Stage compatibility concerns the intended models. It is not a conservativity theorem for recursively axiomatized proof systems, because none has been specified.

5.1 Ranks and limit stages

Define the static language rank of an arithmetic atom to be 0, that of $T_\beta(t)$ to be $\beta + 1$, that of a negation or quantification to be the subformula rank, and that of a conjunction to be the maximum of its two ranks. For a well-formed formula φ over L_θ , φ is an L_α formula exactly when $\text{rk}(\varphi) \leq \alpha$. This rank measures which predicate symbols occur; it need not measure the depth of truth references hidden in a numeral supplied to a predicate.

At a nonzero limit $\lambda \leq \theta$, every L_λ formula uses finitely many earlier predicate symbols, so it already belongs to some L_α with $\alpha < \lambda$. Stage compatibility therefore yields

$$\text{Tr}_\lambda = \bigcup_{\alpha < \lambda} \text{Tr}_\alpha.$$

No infinitary conjunction has been silently added. Successor stages add a new truth predicate to the language; limit stages collect all earlier finite sentences and their stable interpretations.

For example, let φ be $1 + 1 = 1 + 1$. It is true in M_0 , and $T_0(\ulcorner \varphi \urcorner)$ is true in M_1 . The sentence $T_1(\ulcorner T_0(\ulcorner \varphi \urcorner) \urcorner)$ is true in M_2 . At level $\omega + 1$, T_ω can refer to any sentence involving finitely many finite-level predicates. These are semantic examples, not computed decisions of arbitrary arithmetic sentences.

6 Fractal meta-representation: the completeness target

The adjective *fractal* is used here in a syntactic, not geometric, sense. The repeated pattern is that a closed derivation becomes a typed record in the same grammar, its acceptance is again represented by a closed derivation, and that derivation may itself be reified at a later rank. The phrase carries no physical dimension and asserts no fractal dimension. A finite cyclic record may denote an infinite unfolding only after a global validity condition has been specified. Existing cyclic proof systems demonstrate that finite recursive proof graphs and global trace conditions are mathematically workable, while also showing that the trace condition and its soundness proof are substantive parts of the calculus [10], [11].

Definition 6.1 (Ranked self-closing meta-surface). Let Λ be a nonzero limit ordinal. For each $\alpha < \Lambda$, let S_α be a set of well-typed native expressions, let D_α be a set of closed native derivations, and let $c_\alpha : D_\alpha \rightarrow S_\alpha$ give their conclusions. Require cumulative inclusions for earlier ranks, with the conclusion maps preserved, and require

$$S_\lambda = \bigcup_{\alpha < \lambda} S_\alpha, \quad D_\lambda = \bigcup_{\alpha < \lambda} D_\alpha$$

at each nonzero limit $\lambda < \Lambda$. For every α with $\alpha + 1 < \Lambda$, supply a reification map $e_\alpha : D_\alpha \rightarrow S_{\alpha+1}$, an acceptance-forming constructor

$$\text{Acc}_{\alpha+1} : S_{\alpha+1} \times S_\alpha \rightarrow S_{\alpha+1},$$

and an acceptance-derivation map $v_\alpha : D_\alpha \rightarrow D_{\alpha+1}$ satisfying

$$c_{\alpha+1}(v_\alpha(d)) = \text{Acc}_{\alpha+1}(e_\alpha(d), c_\alpha(d)).$$

Such a family is a *ranked self-closing meta-surface*. It is self-closing because each acceptance derivation $v_\alpha(d)$ is eligible for reification at the next rank. Call it *internally self-derivable* relative to native rule systems $(\mathcal{R}_\alpha)_{\alpha < \Lambda}$ when each D_α consists of the closed derivations generated by $\mathcal{R}_\alpha$ and v_α is a rule-defined constructor whose output is checkable as an $\mathcal{R}_{\alpha+1}$ derivation. This is a syntactic condition. Calling its accepted conclusions semantically valid additionally requires a soundness theorem. The successor shift is part of the typing discipline; the definition supplies no same-rank predicate for the full truth of S_α .

An Ordinatics instance must place the translated rank- α arithmetic expressions and claims in S_α . It is Hypermath-derived only when the sets, derivations, reification maps, acceptance constructor, acceptance-derivation maps, and any claimed native rule checkers are constructed from a specified Hypermath grammar and its inference rules. The displayed definition is an abstract interface. The current Hypermath source does not yet supply this complete instance, its arithmetic interpretation, or the soundness theorem for Acc .

Definition 6.2 (Observation-exact reification). Let $\mathcal{D}$ be a set of derivations, $\mathcal{R}$ a set of records, $\mathcal{Q}$ a set of admissible observations, and B a set of outcomes. For an encoding $e : \mathcal{D} \rightarrow \mathcal{R}$ and an observation semantics $b : \mathcal{D} \times \mathcal{Q} \rightarrow B$, say that e is observation-exact for $\mathcal{Q}$ if there is a decoder $\bar{b} : e[\mathcal{D}] \times \mathcal{Q} \rightarrow B$ such that

$$\bar{b}(e(d), q) = b(d, q) \quad (d \in \mathcal{D}, q \in \mathcal{Q}).$$

Theorem 6.3 (Reification factorization criterion). *An observation-exact decoder exists if and only if, for all $d, d' \in \mathcal{D}$ and $q \in \mathcal{Q}$,*

$$e(d) = e(d') \implies b(d, q) = b(d', q).$$

When it exists, the decoder is unique on $e[\mathcal{D}] \times \mathcal{Q}$.

Proof. If a decoder exists, equal records receive equal decoded outcomes, which gives the displayed implication. Conversely, for $r \in e[\mathcal{D}]$ choose any d with $e(d) = r$ and put $\bar{b}(r, q) = b(d, q)$. The implication makes this value independent of the chosen preimage, and it also gives uniqueness. $\square$

Literal recovery of an entire derivation is sufficient for this criterion, but is not necessary. A pointer to a retained derivation preserves its information only because the pointed-to store remains part of the representation. If an encoding collapses records that differ on stage, formula, dependency, branch, or global-trace evidence, then those features cannot be recovered by a decoder. In particular, the value $W(j(\alpha))$ cannot replace the ordinal rank and derivation record; Lemma 4.1 already shows the obstruction when comparable ranks collapse.

Proposition 6.4 (Preservation under finite reuse). *Fix an observation-exact encoding $e : \mathcal{D} \rightarrow \mathcal{R}$ for b . Let $(c_i : \mathcal{D} \rightarrow \mathcal{D})_{i \in I}$ be permitted reuse operations such that*

$$e(d) = e(d') \implies e(c_i(d)) = e(c_i(d')) \quad (i \in I).$$

For every finite composite $c = c_{i_n} \circ \dots \circ c_{i_1}$, including the identity, the observations $(d, q) \mapsto b(c(d), q)$ factor through e .

Proof. Induction on the number of operations shows that $e(d) = e(d')$ implies $e(c(d)) = e(c(d'))$. Observation-exactness then gives $b(c(d), q) = b(c(d'), q)$ for every q . Apply the factorization criterion. $\square$

The premise about each operation is essential to this sufficient condition. For example, projecting a pair of bits onto its first bit preserves first-bit queries and remains adequate under arbitrary first-bit flips. Swapping the bits exposes the discarded second bit: $(0, 0)$ and $(0, 1)$ have the same encoded record but yield different observations after the swap. A correct base decoder alone therefore does not certify reuse. This proposition concerns finite contexts for a fixed encoding. Applying it to ranked self-closing surfaces still requires suitable typed representations and operation laws; it supplies no limit-stage rule or native acceptance constructor.

Proposition 6.5 (Native numeral equality requires separation). *For an encoding $E : \mathbb{N} \rightarrow \mathcal{F}$, put $b(n, k) = 1$ when $n = k$ and $b(n, k) = 0$ otherwise. These equality queries have an exact decoder on $E[\mathbb{N}]$ if and only if E is injective.*

Proof. If $E(n) = E(m)$, the factorization criterion gives $b(n, n) = b(m, n)$. The left side is 1, so $m = n$. Conversely, injectivity makes every fiber contain exactly one input, and hence the criterion holds for these queries. $\square$

These results have checked Lean counterparts in the reviewed Hypermath source [16]. Decoder existence uses classical choice, including when factorization is applied after reuse. The generic lemmas for finite-reuse preservation of equal observations and equivalence of injectivity with the equality-query factorization premise have no axiom dependencies. Specializing $E(n) = \text{f2f}^n(\text{ground})$ uses the existing native parameters without adding a logical clause. In the six-Form model of all 38 declared logical clauses, $E(1) = E(3)$, so no such equality-query decoder exists, although addition and multiplication respect equality of represented values. The model refutes this encoding's adequacy for the original natural-number equality observations. It does not exclude every alternative native encoding. Injectivity, when available, would still leave the native checker, arithmetic operations, quantifiers, and soundness to be connected.

The semantic hierarchy of Section 5 contains no proof predicate or recursively axiomatized progression. The nearest established proof-theoretic comparison starts with Peano arithmetic, abbreviated PA, and iterates full uniform reflection. Schematically, if U is a recursively presented arithmetic theory, its full uniform reflection scheme contains

$$\forall \vec{x} (\text{Pr}_U(\ulcorner \varphi(\vec{x}) \urcorner) \rightarrow \varphi(\vec{x}))$$

for each arithmetic formula φ , where Pr_U is the arithmetized proof predicate and the dots denote numeral substitution. A notation-indexed progression has $U_0 = \text{PA}$, adds this scheme at successor notations, and takes the indicated union at limit notations. This description is schematic until the ordinal-notation system, predecessor relation, limit presentation, proof coding, and accepted well-foundedness evidence are fixed.

Feferman's completeness theorem and its modern sharp bound give the external coverage statement

$$\mathbb{N} \models \varphi \implies \exists a \in \mathcal{O}_K (|a| < \omega^\omega \wedge U_a \vdash \varphi)$$

for every arithmetic sentence φ , where $|a|$ is the ordinal denoted by the Kleene notation a [5], [6]. More precisely, a true Π_{2n+1} sentence can be reached at a notation of order type $\omega^{n+1} + 1$ [6]. This external coverage across a family must be distinguished from stagewise completeness: no fixed sound recursively enumerable stage is thereby claimed to decide all arithmetic truth. It also supplies neither effective decidability nor an effective procedure that selects a valid notation from φ . Membership in $\mathcal{O}_K$ is itself not an arithmetical decision problem. Moreover, the theory reached can depend on the notation and path, not only on the denoted ordinal; Feferman records both the completeness of suitable full-uniform-reflection paths and the failure of invariance across arbitrary paths [7]. The bound explains the use of ω^ω here, but a canonical polynomial code for an ordinal below that bound is not thereby a Kleene notation or a well-foundedness certificate.

The connection between iterated Tarskian truth and reflection is itself established: Beklemishev and Pakhomov analyze transfinitely iterated truth definitions with reflection and canonical ordinal notations, while Leigh characterizes iterated reflection over typed and disquotational truth theories [3], [4]. Reflective Grounded Arithmetic, abbreviated RGA, is still closer to the same-language machinery: recent machine-checked developments internalize proof checking and truth in a paracomplete arithmetic [8], [9]. Its open completeness concerns grounded truth and coexists with ω -incompleteness; it does not claim classical coverage of all standard arithmetic truth. The target here instead retains classical semantics and asks for a path-retaining native realization of the external reflection coverage above.

This comparison gives a precise reading of the project's phrase *Gödelian complete ordinal arithmetic*: sound external coverage of standard arithmetic truth by a transfinite family of stages, with notation and derivation evidence retained. It does not mean decidability, truth definable in its own object language, or a recursively enumerable theory that proves every arithmetic truth.

Theorem 6.6 (Conditional record coverage). *Let $\mathcal{O}$ be an accepted notation class and $(U_a)_{a \in \mathcal{O}}$ have the external coverage property above. Let $\mathcal{A}$ be the set of arithmetic sentences and let*

$$\mathcal{D} = \{(a, p) : a \in \mathcal{O}, p \in \mathbb{N}\}$$

be the set of notation and candidate-proof-code pairs. Define the proof observation $b : \mathcal{D} \times \mathcal{A} \rightarrow \{0, 1\}$ by

$$b((a, p), \varphi) = 1 \iff \text{Proof}_{U_a}(p, \varphi).$$

Let $e : \mathcal{D} \rightarrow \mathcal{R}$ be observation-exact for these sentence queries, with decoder $\bar{b} : e[\mathcal{D}] \times \mathcal{A} \rightarrow \{0, 1\}$. Suppose a native calculus H supplies a translation ι and a record-only checker Check_H such that

$$\text{Check}_H(r, \iota(\varphi)) = \bar{b}(r, \varphi) \quad (r \in e[\mathcal{D}], \varphi \in \mathcal{A}),$$

and suppose the checker is arithmetically sound:

$$\text{Check}_H(r, \iota(\varphi)) = 1 \implies \mathbb{N} \models \varphi \quad (r \in e[\mathcal{D}], \varphi \in \mathcal{A}).$$

Then, for every arithmetic sentence φ ,

$$\mathbb{N} \models \varphi \iff \exists r \in e[\mathcal{D}] \text{ Check}_H(r, \iota(\varphi)) = 1.$$

Proof. For the forward direction, external coverage supplies a notation a and a code p that proves φ . The proof observation therefore equals one. Observation-exactness and checker agreement show that the reified record is accepted for $\iota(\varphi)$. The reverse direction is the checker-soundness assumption. $\square$

The conclusion refers to the native record alone; the external proof code appears only in the construction of that record. Observation-exactness prevents an encoding from merging proof packages that the checker must distinguish. The checker-agreement equation is still a substantive realization obligation, and soundness must be proved independently of the truth it is intended to establish. For ι to be a structure-preserving arithmetic interpretation, it must preserve arithmetic operations, negation, and quantification. A realization as a ranked self-closing meta-surface must additionally place each record and its acceptance derivation at the specified successor ranks. Accepted notation evidence and any cyclic global condition must be checked by stated rules.

If ι is computable, $e[\mathcal{D}]$ is effectively enumerable, and Check_H is uniformly decidable, total sound coverage would decide arithmetic truth by parallel searches for records checking a sentence and its negation. At least one of those effectiveness properties must therefore fail for full classical coverage.

The current Hypermath development supplies finite traces, a proposed self-derivation target, and explicit audit surfaces, but its checked countermodels separate that target from the required arithmetic action and interpretation bridges and refute its entailment from the current clauses [16]. Neither that development nor the Python companion below currently supplies the ranked surface, checker agreement, arithmetic interpretation, or soundness hypothesis of the record-coverage theorem. Their recursively grounded arithmetic completeness status is therefore unresolved, rather than established by the conditional result.

A finite source-syntax realization. The reviewed Hypermath development now constructs a bounded instance of record representation [16]. Let T be the free finite term grammar $t ::= \square \mid \text{apply}(t)$, taken from the formation syntax of the source’s ground layer. Let D_0 contain the ground-self rule and the three parameterized rule schemata $\text{diff}(t)$, $\text{sim}(t)$, and $\text{box}(t)$. Their conclusions are, respectively, that ground continues from itself, that $\text{apply}(t)$ is structurally distinct from ground, that it continues from ground, and that $\text{apply}^2(t)$ structurally orbits t . These are three separate binary predicates; no identification with equality is made.

Encode a ground-self instance as the depth-zero term. For an argument of depth d , encode the other three tags at term depths $4d + 1$, $4d + 2$, and $4d + 3$. The decoder $\delta : T \rightarrow D_0 \cup \{\perp\}$ reverses this assignment and rejects positive multiples of four. Here $\perp$ denotes decoding failure, not a false arithmetic sentence. Define $\text{check}(r, s) = 1$ exactly when r decodes and its decoded instance has syntactic conclusion s . The decoder takes the record alone; the checker additionally takes the claimed statement. This unary code makes no compression claim.

Proposition 6.7 (Primitive records and finite rule reuse). *For this encoding $e : D_0 \rightarrow T$, $\delta(e(p)) = p$ for every $p \in D_0$. Consequently e is injective and every observation of p can be recovered from $e(p)$. In every interpretation satisfying the four primitive source-rule schemata, $\text{check}(r, s) = 1$ implies that s holds. Let u replace each parameterized rule’s argument t by $\text{apply}(t)$ and leave the ground-self instance unchanged. Define R on decodable records by $R(e(p)) = e(u(p))$. Then for every finite n and $p \in D_0$, $R^n(e(p)) = e(u^n(p))$, and the result checks against the conclusion of $u^n(p)$.*

Proof. Depth zero identifies the ground-self rule. Otherwise the depth modulo four identifies the rule tag, and removing that tag and dividing the remaining depth by four recovers the argument’s

depth, hence its unique free term. This proves the round trip and injectivity; composing the decoder with any observation proves observation recovery. An accepted statement is exactly a decoded rule instance's conclusion, so its validity follows from the corresponding rule premise in the interpretation. Induction on n proves the reuse equation, and the round trip gives acceptance of its resulting record. $\square$

The Lean counterparts include `decode_encode`, `check_sound`, and `reuse_many_encode`. The native soundness specialization uses the existing source parameters and four primitive clauses, with no new native axiom, admission, or classical-choice dependency. Lean's inductive types, recursion, propositional extensionality, and quotient equality principle remain explicit host infrastructure. This construction represents primitive instances. The extension below encodes composed derivations; neither construction internalizes the checker or derives its own acceptance statements at a new rank.

Free terms must also be distinguished from their semantic interpretation as `Form` values. In the six-Form model of all 38 source clauses, the depth-one and depth-three records for `diff( $\square$ )` and `box( $\square$ )` have the same interpretation but different syntactic conclusions. Therefore no decoder of that semantic value alone recovers every original instance under this encoding. Both conclusions can hold in the model; the obstruction concerns record recovery, not consistency of the primitive rules. A native reification theorem must preserve the observations required after interpretation. The finite construction does not discharge that obligation or the hypotheses of the conditional coverage theorem.

Composed records and retained formation trees. Extend the finite formula grammar with similarity, negated simulation, and conjunction. A typed derivation is generated by the four primitive schemata, the forward directions of the three source predicate closes, conjunction introduction, and the two projections. The closes infer similarity to ground from continuation to ground, negated simulation from structural distinctness, and similarity of $\text{apply}^2(t)$ to t from its structural orbit. Conjunction has the host logical interpretation already used by the source; adequacy for its native proposition type remains an obligation.

A raw record retains every rule name, term argument, premise record, and projection annotation. Let $K(r, s)$ first check every premise and required annotation, then compare the computed conclusion with s . Let $q(d)$ quote a typed derivation as a raw record, and let $\rho(r)$ reconstruct the typed derivation after successful checking. Reconstruction takes the record alone; its Boolean check supplies the evidence needed by the typed constructor.

Proposition 6.8 (Composed checking and exact record reconstruction). *Every typed derivation d with conclusion $c(d)$ satisfies $K(q(d), c(d)) = 1$. Every accepted record satisfies $q(\rho(r)) = r$. Consequently all observations of its formation tree survive reconstruction. A formula has an accepted record exactly when it is derivable in this finite calculus. Every accepted conclusion holds in each interpretation satisfying the four primitive and three predicate-close premises.*

Proof. Induct on the typed derivation for acceptance and soundness. Primitive cases use their corresponding source clauses, closes use the stated forward implications, and conjunction rules use introduction or elimination. For reconstruction, induct on the raw record. Its acceptance check gives accepted premises with the required conclusions; reconstruct them inductively and apply the indicated typed rule. Quoting that result returns the same rule, arguments, annotations, and premise records. The two constructions give the representability equivalence. Composing the exact round trip with any function of a record proves observation preservation. $\square$

The equivalence is checker coverage of the specified calculus, not semantic completeness for its models or arithmetic truth. Checking a projection includes its entire conjunction premise; a valid selected branch cannot hide a failed unselected branch.

Single-term codes and their cost. Represent each constructor and its arguments by a tagged finite binary tree. Encode a leaf by bit 0, and a fork by bit 1 followed by its two child codes. The parser must consume the complete input. Pack a bit list into a natural number by $P(\square) = 0$, $P(0 :: b) = 2P(b) + 1$, and $P(1 :: b) = 2P(b) + 2$. Tag the outer record and formula trees differently. Write $E(r)$ and $F(s)$ for the resulting numbers; the corresponding free ground terms have those depths. Let $\widehat{K}$ decode both numbers and then run K , rejecting either decoding failure.

Proposition 6.9 (Encoded composed checking). *The record and formula encodings have exact decoders, hence are injective in their respective sorts. Every observation of a record is recoverable from its single free ground term, and*

$$\widehat{K}(E(r), F(s)) = K(r, s).$$

For a prefix of length b with packed value n , $2^b \leq n + 1$ and $n + 2 \leq 2^{b+1}$.

Proof. The first packed bit is determined by parity, and removing it strictly decreases the remaining number. Induction gives unpacking after packing. Structural induction on a tree gives the prefix-parser round trip; requiring empty remaining input excludes trailing data. Constructor tags then give record and formula recovery by structural induction, including all premises and annotations. The free term of a given depth is unique. Recovery proves injectivity and observation preservation; the definition of $\widehat{K}$ gives checker agreement. Induction on the bit list proves the two bounds. $\square$

Thus materializing the unary term is exponential in the prefix length. The executable interface works on packed numbers: one checked separation record has 81 prefix bits but unary depth 2820815200072616987372202. This is not a compression or performance theorem. Repeated subrecords remain stored in full. The Lean dependency report exposes classical choice in the size-bound proof; the executable decoder and checker do not use it.

Proposition 6.10 (Faithful model and an operational boundary). *There is a model of all 38 current logical clauses in which the composed record and formula codes retain their full decoded objects and encoded checking agrees with K . In that same model, the current finite congruence-preserving path relation cannot carry any encoded record to any encoded formula.*

Proof. Use the existing two-chain model, with forms (n, ε) for $n \in \mathbb{N}$ and $\varepsilon \in \{0, 1\}$, ground $(0, 0)$, and application $(n, \varepsilon) \mapsto (n + 1, \varepsilon)$. Congruence and simulation are equality; similarity is universal. The remaining parameter interpretations and all 38 clause checks are explicit in `FullAxiomModel.lean` [16]. The record and formula values are $(E(r), 0)$ and $(F(s), 0)$, exactly the interpretations of their free ground terms. Reading their first coordinates gives the required decoders and checker; values on the second chain are rejected.

The current preserving edge from x to y requires both $y = \text{apply}(x)$ and congruence of y with x . Here that would require $n + 1 = n$, so every preserving path is empty and has equal endpoints. The distinct record and formula envelopes give unequal values. No such path can therefore perform their direct transition. $\square$

The model is a compatibility witness, not a proof that all interpretations are faithful or that it captures the intended native semantics. Its decoders and checker are host functions on model

values. The path obstruction concerns that direct transition in this specified model; it does not exclude a different native computation mechanism or ranked acceptance construction. Together with the collapsing model, it separates possible record retention from retention entailed by the clauses. Internal acceptance, source adequacy, arithmetic interpretation, transfinite realization, and coverage remain open.

Proposition 6.11 (Native form closure does not certify a record). *In the model of Proposition 6.10, every encoded raw record is generated from ground and satisfies the source’s syntax, substance, semantics, form-closure, ground-anchored definition, and discharge conditions. Nevertheless there are two records with the same true, derivable conclusion and opposite checker results. No test of only their conclusions can agree with K on every record.*

Proof. Let c assert that ground structurally continues from itself. Set $g = \text{primitive}(\text{groundSelf})$ and $b = \text{projectLeft}(c, c, g)$. Both claim c , but g proves c whereas the projection in b requires a premise proving $c \wedge c$. Thus $K(g, c) = 1$ and $K(b, c) = 0$.

The model’s four unary executive predicates hold on all forms. Each record value $(E(r), 0)$ is reached from ground by $E(r)$ applications. The definition and discharge clauses then give the remaining ground-anchored conditions. They hold for b as well as g , and c is true and witnessed by g . A function of the common conclusion and the claim c receives identical inputs for these two records, so it cannot return both checker values. $\square$

This is a failure to certify a particular inference tree, not a false conclusion or an inconsistency. Its checked counterpart retains all 38 clauses and adds no native axiom. Acceptance must bind each rule application and its premises; closure of the represented entity is insufficient.

Execution of the represented finite calculus. To specify the computation that native rules must realize, compile each record r to a postorder instruction sequence $P(r)$. There are seven instruction kinds: primitive introduction, the three predicate closes, conjunction, and the two projections. A state is a stack of formulas or a failure state $\perp$; the successful empty stack is not $\perp$. A primitive pushes its instantiated conclusion. Each close or projection checks the exact required top formula and replaces it with its conclusion. Conjunction consumes the two premises in order. A mismatch or stack underflow produces $\perp$, which every subsequent instruction preserves. These transitions do not call K or receive an external proof object.

Proposition 6.12 (Exact execution and trace acceptance). *Write $c(r)$ for the computed conclusion and $V(r) = K(r, c(r))$ for validity. For every record r and stack Γ ,*

$$\text{run}(P(r), \Gamma) = \begin{cases} c(r) :: \Gamma, & V(r) = 1, \\ \perp, & V(r) = 0. \end{cases}$$

Consequently the machine accepts a claim s exactly when $K(r, s) = 1$. Let $\tau(r)$ list every resulting state when $P(r)$ runs from the empty stack. A submitted trace t is accepted for (r, s) exactly when $t = \tau(r)$ and $K(r, s) = 1$. There is one instruction and one resulting state per retained record node.

Proof. Induct on r , retaining an arbitrary surrounding stack. Each primitive case pushes its conclusion. Each unary case first executes its premise, then performs precisely the annotation test in V . For conjunction, both premise programs execute before their conclusions are combined. A failed premise remains failed in every case. This proves the displayed equation, including invalid records. Comparing the final stack with the singleton $[s]$ gives agreement with K , and hence its relative soundness from Proposition 6.8.

Replay compares each submitted state with the next state prescribed by its instruction, rejecting missing or extra states. Induction on the instruction sequence makes the replayed trace unique. Requiring its final state to be $[s]$ gives the trace-acceptance equivalence. Structural induction on r counts one instruction per node; execution records one state per instruction. $\square$

A faithful trace of a failed run passes replay but fails acceptance. State traces alone need not identify records: projecting either side of $c \wedge c$ can produce the same trace from distinct valid records. The record or instruction identity must therefore accompany the states. The node count excludes formula comparison, packed decoding, program construction, and storage of whole stacks; it is not a time or compression bound.

These statements are checked in `RecordMachine.lean` [16]. Its exact dependency reports use no admission, classical choice, or native Hypermath axiom. They establish host computation, using inductive data, equality, lists, and recursion. A native realization must represent the instructions and states, justify their transitions from specified source rules, and derive an acceptance formula that can itself be reified at the next rank. The source kernel’s proposed identification of substituted rule application with unary application still needs a translation retaining the rule, substitution, premises, and conclusion. This computation and these obligations supply neither an arithmetic inference rule nor a transfinite coverage proof.

The unresolved self-derivation cycle. The exact native target must be distinguished from a proved theorem. Let $\mathcal{A}$ be the 38 logical clauses of the reviewed Hypermath translation, with its declared parameters and finite path definitions [16]. Write g for ground, f for form application, and d for the deriver parameter. Let SD be the original four-conjunct target: ground self-continuation, reflexive finite self-reads, the two path-composition identity laws, and $\text{Cyc} := \text{Simulation}(f(f(d)), d)$.

Proposition 6.13 (The missing simulation cycle). *Relative to $\mathcal{A}$, one has $\text{SD} \leftrightarrow \text{Cyc}$. There are interpretations of all clauses of $\mathcal{A}$ in which SD is false and interpretations in which it is true. The clauses also do not entail the nontrivial simulation-pair claim*

$$\exists x, y \text{ (Similar}(x, y) \wedge \text{Simulation}(x, y) \wedge x \neq y).$$

Proof. Ground self-continuation is a clause of $\mathcal{A}$; every form has its zero-step self-read; and path composition with the ground path on either side satisfies the stated identity clauses. These establish the first three conjuncts of SD. Its last conjunct gives the forward implication, and adjoining it to the first three gives the reverse implication.

In the two-chain interpretation of Proposition 6.10, $d = g = (0, 0)$ and simulation is equality. Thus $f(f(d)) = (2, 0) \neq (0, 0)$, so Cyc and SD are false. Simulation as equality also refutes the displayed nontrivial-pair claim. The same interpretation still satisfies every clause of $\mathcal{A}$.

For the positive interpretation, use the six-form model in `FiniteActionCountermodel.lean` [16]. Its forms are $a_0, a_1, a_2, b_0, b_1, b_2$, and application has the chains $a_0 \mapsto a_1 \mapsto a_2 \mapsto a_1$ and $b_0 \mapsto b_1 \mapsto b_2 \mapsto b_0$. Ground is a_0 , deriver is a_1 , and simulation is equality. Congruence is equality of the class labels 0, 1, 2, 3, 1, 2 respectively; similarity is universal. The remaining interpretations and all 38 clause checks are given in that model. Here $f(f(d)) = d$, so Cyc holds and the first part gives SD. $\square$

The equivalence is checked by `selfDerivation_iff_driverCycleClaim`; its conditional assembly uses existing clauses and parameters, with no admitted proof. The three negative results in `FullAxiomModel.lean` have no axiom dependencies. Certifying that the complete model satisfies $\mathcal{A}$ uses Lean’s propositional extensionality, classical choice, and quotient soundness. The models concern the present translation, whose adequacy to the intended source remains open.

The six-form model also refutes the proposed arithmetic bridges discussed above. Thus establishing the cycle would still not prove those bridges. The foundation now retains `selfDerivation` as the original unproved proposition and exposes conditional assembly separately. Five admitted proof sites for the cycle and nontrivial pair were withdrawn; their removal does not prove the claims. A source-adequate simulation and generation account must justify any rule excluding the countermodel. Assuming the desired cycle as an axiom would assume the remaining conjunct. Native ranked acceptance and arithmetic coverage need their own constructions beyond that obligation.

7 Why a truth stage cannot certify its own full truth

Theorem 7.1 (Undefinability at each stage). *For every $\alpha \leq \theta$, there is no L_α formula $\tau(x)$ that defines Tr_α in M_α .*

Proof. The arithmetic part represents the syntactic substitution operation for the fixed effective language coding. Apply the diagonal lemma to $\neg\tau(x)$ to obtain an L_α sentence λ satisfying

$$M_\alpha \models \lambda \leftrightarrow \neg\tau(\ulcorner\lambda\urcorner).$$

If τ defined Tr_α , the same structure would satisfy $\tau(\ulcorner\lambda\urcorner) \leftrightarrow \lambda$. Together these force $\lambda \leftrightarrow \neg\lambda$, impossible in the classical two-valued structure. The diagonal lemma’s syntactic construction uses only finitely many symbols of the candidate formula, so the countable predicate vocabulary causes no problem [13]. $\square$

For $\alpha < \theta$, the next predicate T_α is not in L_α , which is exactly why typed disquotation is compatible with this theorem. A sentence in $L_{\alpha+1}$ may apply T_α to its own code, but if that sentence uses T_α , its code is outside $\text{Sent}(L_\alpha)$ and the predicate returns false under our convention. Such an application is not a correct assertion of that sentence’s own truth.

A precise positive answer. The ordinal domain gives a definable copy of arithmetic; the metatheory gives its satisfaction relation; and the ordinal hierarchy provides successively richer languages in which earlier truth is available as a predicate. These together furnish the advertised sense of Tarski-definable arithmetic. The theorem rules out treating the union stage, the wrap map, or the name “ordinal” as an unrestricted self-truth mechanism.

The result is a semantic construction and assumes the resources of the metatheory. It supplies neither a recursively enumerable complete theory of arithmetic nor a proof of that metatheory’s consistency. Allowing a source-specific similarity relation between a sentence and its negation would require a different logical semantics; it would not be a proof of the classical biconditionals used here.

8 Computational companion

The accompanying Python library, `ordinatics` development version 0.2.0.dev0, implements a computable fragment of the construction and its algebraic interfaces.¹ It uses exact integer coefficient tuples for ordinals below ω^ω . The operators `+` and `*` implement ordinary ordinal operations; `natural_add` and `natural_mul` implement the commutative polynomial operations. Conversion through `to_sympy` makes the representation map j explicit. The resulting expressions can be used with SymPy symbolic calculus, matrices, and solvers, or converted to numerical functions for NumPy and SciPy. Numerical evaluation of a polynomial image is distinct from ordinal arithmetic.

¹<https://github.com/TimeLordRaps/ordinatics>

The function `rational_function` validates and reduces exact rational functions. The functions `specialize` and `wrap` evaluate the reduced forms and raise a distinct pole exception when the reduced denominator vanishes at the evaluation point. Thus a removable singularity and a pole are observably different. A fixed logarithm branch is used for rational-power images. These routines realize the bounded value construction rather than a total evaluation of the whole field.

For semantics, the library supplies immutable arithmetic term and formula trees, static ordinal ranks, and an iterative evaluator. The constructors `Exists` and `ForAll` require a bound term and range over $0 \leq n < b$; the bound is evaluated in the outer assignment, and the quantifier binds its variable only in the body. A valid `Truth(beta, sentence)` node quotes a closed formula of rank at most `beta`; its language rank is $\beta + 1$. The `rank` and `evaluate` functions enforce these conditions before evaluation. For example:

```
from ordinatics import Add, Eq, Nat, Truth, ZERO, ONE, evaluate
phi = Eq(Add(Nat(1), Nat(1)), Nat(2))
assert evaluate(Truth(ZERO, phi), stage=ONE)
```

This quotation interface operates on explicit finite syntax trees. It is not an implementation of unrestricted numerical sentence codes. Invalid quotations are rejected before evaluation, whereas the mathematical truth sets defined above exclude malformed numerical codes by convention. The difference is part of the interface contract.

Ordinal labels enforce language membership. Evaluation of a valid quotation directly evaluates its closed quoted tree; it does not execute through preceding transfinite stages or construct their truth sets. Consequently the quotation nodes supply a typing discipline within this computable fragment, not an oracle for the full hierarchy.

Every quantifier iteration and tree traversal is charged against an explicit work-step budget. Exhaustion raises an exception rather than returning false. The budget bounds these steps, not the bit complexity of arbitrarily large integers or the execution time of external symbolic routines. The implementation does not construct Tr_α for the full unbounded language. Its executable claims concern bounded formulas and exact algebraic examples; software tests do not replace the proofs of the semantic theorems.

9 Contribution, limitations, and the next formal problem

The model joins three components with distinct contracts. The ordinal layer of *Ordinatics* supplies ordered values, finite arithmetic, and semantic stage indices. The bounded field layer supplies division, while `wrap` evaluation is restricted to functions regular at the chosen anchor. The satisfaction hierarchy supplies meaning for truth claims at specified language levels. None of these components obtains the other's authority merely by sharing a representation.

The dependency structure is deliberately revealing. The truth-hierarchy theorems require ordinal well-foundedness, syntax, and set-theoretic satisfaction, but they do not require the anchor $-1/2$, fractional powers, or a complex spiral. Thus the paper establishes a coherent interface *for* *Ordinatics* rather than proving that the value layer of *Ordinatics* is necessary for arithmetic truth. A stronger claim of necessity would require an additional theorem showing a semantic construction that depends essentially on that layer. No such theorem is asserted here.

The source's path-sensitive viewpoint has a concrete consequence nonetheless: semantic records should retain the formula, its language, its ordinal level, and its interpretation, even when numerical evaluations coincide. A verification record could therefore contain $(\alpha, \ulcorner \varphi \urcorner, s, v, p)$, where v is a claimed satisfaction value and p is a proof or evaluation certificate in a separately specified calculus.

The existence theorem does not generate p . For an implementation, failure to obtain such a certificate must remain an unestablished claim, rather than being converted into either truth or falsity.

Four formal questions remain. First, a derivation from the $\square$ axioms needs a defined syntax, inference rules, and an interpretation theorem that actually produces the ordinal domain and the required recursion. Second, a complete treatment of Ordinatrics needs a specified field or other algebra, a coherent root convention, and exact partiality conditions for wrap. Third, beyond the bounded evaluator, a proof-certificate checker needs a specified calculus and a soundness theorem relating accepted certificates to the displayed satisfaction clauses. Fourth, the intended fractal meta-representation must realize a named reflection progression, preserve the observations used after recursive reuse, and bind notation presentations to well-foundedness evidence. These questions are separable and permit bounded progress without claiming a universal truth algorithm.

The mathematically supportable thesis is therefore that ordinal structure can organize a typed foundation for arithmetic satisfaction while preserving both operational distinctions and semantic limits. The present reconstruction makes that thesis explicit. Its elementary obstructions and constructive model are offered as a technical foundation note; independent mathematical review and a broader novelty assessment remain appropriate before a stronger research claim.

References

- [1] Alfred Tarski. The Semantic Conception of Truth and the Foundations of Semantics. *Philosophy and Phenomenological Research* 4(3), 341–376, 1944. [doi:10.2307/2102968](https://doi.org/10.2307/2102968). Author’s article available as a [web transcription](#).
- [2] Volker Halbach. *Axiomatic Theories of Truth*. Second edition, Cambridge University Press, 2014. [doi:10.1017/CBO9781139696586](https://doi.org/10.1017/CBO9781139696586).
- [3] Lev D. Beklemishev and Fedor N. Pakhomov. Reflection Algebras and Conservation Results for Theories of Iterated Truth. *Annals of Pure and Applied Logic* 173(5), 103093, 2022. [doi:10.1016/j.apal.2022.103093](https://doi.org/10.1016/j.apal.2022.103093).
- [4] Graham E. Leigh. Reflecting on Truth. *IfCoLog Journal of Logics and their Applications* 3(4), 557–594, 2016. [Author-hosted journal issue](#).
- [5] Solomon Feferman. Transfinite Recursive Progressions of Axiomatic Theories. *The Journal of Symbolic Logic* 27(3), 259–316, 1962. [doi:10.2307/2964649](https://doi.org/10.2307/2964649).
- [6] Fedor Pakhomov, Michael Rathjen, and Dino Rossegger. Feferman’s Completeness Theorem. [arXiv:2405.09275 \[math.LO\]](https://arxiv.org/abs/2405.09275), 2024; published in *Bulletin of Symbolic Logic* 31(3), 462–487, 2025, [doi:10.1017/bsl.2025.2](https://doi.org/10.1017/bsl.2025.2).
- [7] Solomon Feferman. Turing’s Thesis. *Notices of the American Mathematical Society* 53(10), 1200–1205, 2006. [Author-hosted article](#).
- [8] Bryan Ford. Computable Quantification in Reflective Grounded Arithmetic. [arXiv:2607.25533 \[math.LO\]](https://arxiv.org/abs/2607.25533), 2026.
- [9] Bryan Ford. Internalized Truth in Reflective Grounded Arithmetic. [arXiv:2608.16140 \[math.LO\]](https://arxiv.org/abs/2608.16140), 2026.
- [10] James Brotherston and Alex Simpson. Sequent Calculi for Induction and Infinite Descent. *Journal of Logic and Computation* 21(6), 1177–1216, 2011. [doi:10.1093/logcom/exq052](https://doi.org/10.1093/logcom/exq052).
- [11] Anupam Das. On the Logical Complexity of Cyclic Arithmetic. *Logical Methods in Computer Science* 16(1:1), 1–39, 2020. [doi:10.23638/LMCS-16\(1:1\)2020](https://doi.org/10.23638/LMCS-16(1:1)2020).
- [12] Lorenz Halbeisen. The Axioms of Set Theory ZFC, Chapter 13, especially Theorem 13.3 and the ordinal arithmetic section. [Author-hosted chapter](#), accessed September 6, 2026.
- [13] Michael Beeson. Lecture 13: The First Incompleteness Theorem. Stanford logic lecture slides, especially the self-reference lemma and undefinability discussion. [Author-hosted slides](#), accessed September 6, 2026.

- [14] The Stacks Project Authors. Section 10.9: Localization. [Tag 00CM](#), accessed September 7, 2026.
- [15] National Institute of Standards and Technology. Digital Library of Mathematical Functions, Section 4.2: Definitions, Logarithm, Exponential, Powers. [DLMF Section 4.2](#), accessed September 7, 2026.
- [16] Tyler Roost. *Hypermath: Fractal meta-representation and Gödelian completeness research target*. Development source and proof audit at commit [b35b795c13390e5e10430238e033955ca5053847](#), 2026. The repository records open proof obligations and countermodels; it is not an independent completeness certificate.
- [17] Tyler Roost. Unpublished working notes on Ordinalics: *Ordinalics* (Chapter 48), *The Wrap Operation W* (Chapter 50), *The Four-Level Ordinate Hierarchy* (Chapter 97), and *Dimensional Transition Operators* (Chapter 98), with accompanying ordinal-extension specifications. Manuscripts consulted September 6, 2026; Chapters 97–98 dated August 6, 2026. These are sources of the research proposal, not independently validated proof certificates. All definitions required for the present results are given in this paper.